1 Memory Technologies on a Microcontroller

A microcontroller contains two fundamentally different kinds of on-chip memory, and every part of a C program can live in only one of them. The choice is dictated by three properties:

1. Whether the memory sits on the CPU's memory bus (so every byte or word has an address),
2. whether it can be written with an ordinary store instruction
3. whether it keeps its content at power-off.

1.1 SRAM, NOR Flash, and Block Devices

SRAM (static RAM) is **connected directly to the memory bus**:

1. every byte has an address and is read or written with a normal load or store
2. **Fast**: one access takes about one clock cycle
3. **Byte-writable** at any time,
4. **volatile**: at power-off its content is lost, and at power-on it is **undefined** (random bits, not zeros). SRAM is therefore the only place where anything that changes at run time can live: variables and the stack.

NOR flash is also on the memory bus, like a ROM:

1. Every word has an address and random-access reads are fast.
2. The CPU can therefore fetch instructions directly from it, which is called **execute-in-place (XIP)**: no copy is needed before running.
3. **Non-volatile**.
4. **Writing is not a store instruction**: a whole sector (a few KB) must first be **erased**, setting all its bits to 1, and then individual words are **programmed**, which can only turn bits from 1 to 0. Both operations take microseconds to milliseconds and go through a command sequence sent to the flash controller, so at run time NOR flash is effectively **read-only**.

NAND flash (SD cards, SSDs) and hard disks are **not** on the memory bus:

1. Accessed through a controller one block (page) at a time, typically 512 B to 4 KB, and a block has no CPU address.
2. The CPU can neither fetch instructions from it nor read a single variable: content must first be copied into RAM. This is why a desktop OS needs a **loader** that copies the executable from the SSD into RAM. A microcontroller has no loader: its program already sits in NOR flash, at the address it runs from.

Technology	On the bus?	Execute from it?	Write at run time?	Keeps data at power-off?
SRAM	yes	yes	yes, bytes	no (volatile)
NOR flash	yes	yes (XIP)	no (erase/program)	yes
NAND / disk	no (blocks)	no	through a driver	yes

1.2 Placement Rules for a C Program

These properties fix where each part of a C program goes:

Program element	Memory	Reason
Code	Flash	executed in place (XIP)
Constants (`const`, string literals)	Flash	never modified: read in place
Variables	RAM	modified at run time
Initialized variables (`int x = 5;`)	RAM at run time + Flash in the image	RAM is random at power-on: the initial value is stored in flash and copied into RAM by the startup code before `main()` (see 2.2.1)

1.3 Cortex-M Address Space

The CPU sees a single 4 GB address space (32-bit addresses). On every Cortex-M the **purpose** of each region is fixed by the ARM architecture; each chip vendor decides how much memory or which devices populate it.

Address range	Region	On `mps2-an385`
`0x0000_0000` – `0x1FFF_FFFF`	Code	4 MB at `0x0`: the program ("FLASH")
`0x2000_0000` – `0x3FFF_FFFF`	SRAM	4 MB at `0x2000_0000`: "RAM"
`0x4000_0000` – `0x5FFF_FFFF`	Peripherals	UART, timers, …
`0x6000_0000` – `0xDFFF_FFFF`	External RAM / devices	not used
`0xE000_0000` – `0xE00F_FFFF`	Private peripheral bus	registers of the CPU core itself (System Control Space at `0xE000_E000`)

The same map, with the populated parts drawn inside the architectural regions:

Cortex-M3 address space on mps2-an385



Peripherals are controlled by reading and writing their addresses (memory-mapped I/O), and the last region holds registers that configure the CPU core itself.

> **INFO**
>
> Emulated flash On `mps2-an385` (and in QEMU) the 4 MB at address `0x0` is fast RAM preloaded with the program image. It plays the role of flash and is treated as read-only, exactly as real flash on a microcontroller.

2 Linker Script and Image Layout

A **linker script** is a text file, passed to the linker with `-T`, that describes the memory layout of the target chip and how the program must be placed in it.

```
MEMORY
{
    FLASH (rx)  : ORIGIN = 0x00000000, LENGTH = 4M
    RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 4M
}

ENTRY(Reset_Handler)

_estack = ORIGIN(RAM) + LENGTH(RAM);
_Min_Stack_Size = 0x1000;
_Min_Heap_Size = 0x1000;

SECTIONS
{
    .isr_vector :
    {
        . = ALIGN(4);
        KEEP(*(.isr_vector))
        . = ALIGN(4);
    } > FLASH

    .text :
    {
        . = ALIGN(4);
        *(.text*)
        *(.rodata*)
        . = ALIGN(4);
        _etext = .;
    } > FLASH

    _sidata = LOADADDR(.data);

    .data :
    {
        . = ALIGN(4);
        _sdata = .;
        *(.data*)
        . = ALIGN(4);
```

```
        _edata = .;
    } > RAM AT > FLASH

    .bss :
    {
        . = ALIGN(4);
        _sbss = .;
        *(.bss*)
        *(COMMON)
        . = ALIGN(4);
        _ebss = .;
    } > RAM

    .heap :
    {
        . = ALIGN(8);
        PROVIDE( end = . );
        PROVIDE( _end = . );
        _heap_bottom = .;
        . = . + _Min_Heap_Size;
        _heap_top = .;
    } > RAM

    __StackLimit = _estack - _Min_Stack_Size;
    ASSERT(__StackLimit >= _heap_top, "region RAM overflowed: heap collides with the stack")
}
```

The linker alone only merges the sections of the object files and resolves symbols: it **does not know which memories the chip has** or where they are. The linker script supplies this information. Based on it, the linker:

- assigns a final address to every section and symbol;
- places each section in the right memory (code and constants in flash, variables in RAM);
- defines symbols that the startup code uses, such as section boundaries and the initial stack pointer;
- checks that everything fits in the available memory, failing the link otherwise.

On bare metal there is **no OS loader** to decide where the program goes. The image must already match the hardware: for example, the vector table must be at address `0x0`, and initialized data must be stored in flash and used in RAM. The linker script is what guarantees this.

Internally, a linker script is divided into four parts:

Part	Role
`MEMORY { ... }`	declares the available memory regions: name, start address (`ORIGIN`), size (`LENGTH`), attributes (`r` , `w` , `x`)

Part	Role
`SECTIONS { ... }`	builds the output sections from the input sections of the object files and assigns each one to a memory region
`ENTRY(...)`	names the symbol where execution starts; used by tools (debugger, QEMU), not by the CPU
Symbol assignments (`_estack = ...;`)	create named addresses that the code can use, such as the top of the stack or the bounds of `.data` and `.bss`

2.1 MEMORY: Regions and Size Checking

```
MEMORY
{
    FLASH (rx)  : ORIGIN = 0x00000000, LENGTH = 4M
    RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 4M
}

_estack = ORIGIN(RAM) + LENGTH(RAM);   /* = 0x20400000 */
```

Each line of `MEMORY` defines one memory region; the two regions correspond to the two populated rows of the memory map (1.3).

Field	Meaning	Example
Name	freely chosen identifier, not a keyword	`FLASH` , `RAM`
Attributes	allowed access: `r` read, `w` write, `x` execute	`(rx)` , `(rwx)`
`ORIGIN`	start address of the region	`0x00000000` , `0x20000000`
`LENGTH`	size of the region	`4M`

The linker uses `LENGTH` to verify that the program fits:

- if the content assigned to a region does not fit, the link fails (e.g. `region 'FLASH' overflowed by 28 bytes`) instead of producing an image that cannot work;
- flash must hold `text + data` as reported by `arm-none-eabi-size` : code and constants, plus the stored initial values of `.data` ;
- `.bss` needs no flash, since it has no stored content;
- example: with `text = 96` and `data = 32` , the minimum FLASH `LENGTH` is exactly 128.

`_estack`. Defined as `ORIGIN(RAM) + LENGTH(RAM)` = `0x20400000` , one byte past the last RAM byte. The stack starts there because on Cortex-M it grows **downwards**: starting at the top of RAM gives it the most room.

2.2 SECTIONS: Output Sections Built from Input Sections

Each object file (`.o`) produced by the compiler or assembler contains its own sections, called **input sections**: `.text` (code), `.rodata` (constants), `.data` , `.bss` . The linker merges them into the sections of the final ELF

file, called **output sections**. The `SECTIONS` block contains one definition per output section, always with the same format:

```
<output_section> :
{
    <rule>            /* which input sections to take, e.g. *(.text*) */
    <rule>            /* rules are applied in order                   */
    <symbol> = .;     /* optional: give a name to the current address */
} > <REGION>          /* memory region where the section is placed    */
```

The output sections defined in this linker script are:

Output section	Content	Region
`.isr_vector`	vector table: initial SP and handler addresses	FLASH, at `0x0`
`.text`	code and read-only data (`.rodata`: constants, string literals)	FLASH
`.data`	initialized global variables	RAM (initial values stored in FLASH)
`.bss`	global variables initialized to zero or not initialized	RAM (zeroed at boot, nothing stored)
`.heap`	space reserved for dynamic allocation (`malloc`)	RAM (only reserved, empty)

The first two, both placed in FLASH, are:

```
.isr_vector :
{
    . = ALIGN(4);
    KEEP(*(.isr_vector))
} > FLASH

.text :
{
    *(.text*)            /* code from every .o          */
    *(.rodata*)          /* constants, string literals  */
    _etext = .;
} > FLASH
```

Element	Meaning
`.text :`	name of the output section created in the final ELF
`*(.text*)`	from every input file (`*`), take every section whose name starts with `.text`
`*(.rodata*)`	same, for the read-only data of every input file
`KEEP(...)`	always keep these input sections, even with garbage collection enabled

Element	Meaning
`.`	**location counter**: the address where the linker is currently placing content
`. = ALIGN(4)`	round the location counter up to a multiple of 4
`_etext = .;`	define a symbol equal to the current address (here, the end of the code)
`> FLASH`	place the output section in the FLASH region

Three consequences follow from these rules:

- **Order of rules = order in memory**: inside `.text`, all the code comes first, then all the read-only data.
- `.rodata` **has no output section of its own**: it is folded into `.text`.
- `.isr_vector` **lands at** `0x0`: it is the first output section placed in FLASH, and `.text` follows right after it.

2.2.1 Run Address (VMA) and Load Address (LMA)

```
_sidata = LOADADDR(.data);
.data :
{
    _sdata = .;       /* start of .data in RAM */
    *(.data*)
    _edata = .;       /* end of .data in RAM   */
} > RAM AT > FLASH
```

Every output section has two addresses:

Address	Meaning	In the script
VMA (virtual memory address, *run address*)	where the program uses the section at run time	`> REGION`
LMA (load memory address)	where the section's bytes are stored in the image	`AT > REGION` (if omitted, LMA = VMA)

For most sections the two coincide; `.data` is the exception:

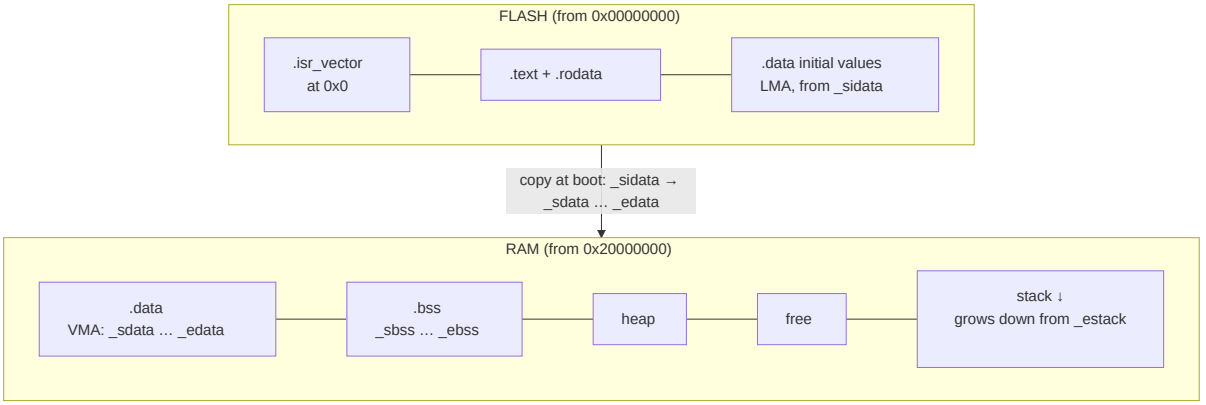
Section	VMA	LMA	Why
`.text`	FLASH	FLASH	code runs where it is stored (XIP)
`.data`	RAM	FLASH	variables are modified in RAM, but their initial values must survive power-off in flash

`> RAM AT > FLASH` means: VMA in RAM, LMA in FLASH. The script also records three addresses for the boot code:

Symbol	Value	Role in the copy
`_sidata`	`LOADADDR(.data)`: start of the initial values in FLASH (LMA)	source
`_sdata`	start of `.data` in RAM (VMA)	destination start
`_edata`	end of `.data` in RAM (VMA)	destination end

> **IMPORTANT**
>
> Nothing copies LMA to VMA automatically The linker only records the two addresses, and the CPU copies nothing at reset. The startup code must copy `.data` from `_sidata` to `_sdata` … `_edata`.

The resulting layout is:



2.2.2 .bss, Heap, and Stack Reservation

These three RAM areas occupy **no space in flash**: the image records only their size or bounds, never their bytes.

- `.bss`: storing zeros in flash would be wasteful, so only the size is recorded.
- `.heap`: it has no initial content at all; it is just space.
- **stack**: it is not even a section, only an address range below `_estack`.

```
.bss :
{
    _sbss = .;
    *(.bss*)
    *(COMMON)
    _ebss = .;
} > RAM

.heap :
{
    . = ALIGN(8);
    _heap_bottom = .;
    . = . + _Min_Heap_Size;
    _heap_top = .;
} > RAM

__StackLimit = _estack - _Min_Stack_Size;
ASSERT(__StackLimit >= _heap_top, "region RAM overflowed: heap collides with the stack")
```

Area	What it is	Bounds	Who prepares it
`.bss`	variables that start at zero	`_sbss` … `_ebss`	boot code fills it with zeros
`.heap`	space reserved for dynamic allocation (`malloc`)	`_heap_bottom` … `_heap_top` (4 KB)	nobody: it is only reserved
stack	**not a section**: the top 4 KB of RAM	`__StackLimit` … `_estack`	SP is set to `_estack` at boot

Element	Meaning
`*(COMMON)`	globals declared without a value (`int counter;`) that some compilers put in a "common" pool: they belong in `.bss` too
`. = . + _Min_Heap_Size`	advance the location counter by 4 KB, so nothing else can be placed there
`__StackLimit`	`_estack - _Min_Stack_Size` : the lowest address the stack may reach
`ASSERT(cond, msg)`	make the link fail with `msg` if `cond` is false

IMPORTANT

Heap–stack collision caught at link time The `ASSERT` makes the **link fail** if the heap would run into the stack limit. A memory corruption that would appear at run time, and be very hard to debug, becomes a clear error at build time.

EXAMPLE

RAM `LENGTH = 8K` `_estack` moves to `0x20002000` and `__StackLimit` to `0x20001000` . The sections `.data` + `.bss` + `.heap` still fit inside 8 KB, so there is no region overflow. However, the 4 KB heap now crosses `__StackLimit` , and the link fails because of the `ASSERT` .

2.2.3 Linker-Defined Symbols

Symbols such as `_sdata` , `_edata` , `_sidata` , `_sbss` , `_ebss` and `_estack` have **no storage**: no memory cell holds their value. The linker only assigns each one an **address**, and the address itself is the information.

```
extern uint32_t _sdata, _edata;   /* "there is something at this address" */
uint32_t *start = &_sdata;        /* correct: the address (0x20000000)    */
uint32_t  oops  = _sdata;         /* wrong: reads RAM at that address     */
```

They are declared as `extern` objects only so that C allows taking their address with `&` . Reading `_sdata` itself would read whatever happens to be in RAM at `0x20000000` . In assembly, `ldr r0, =_sdata` means `r0 = &_sdata;` . The boot code uses these addresses as loop bounds.

2.2.4 Orphan Sections

GCC lets the programmer choose the input section of a variable or function explicitly:

```
int tagged __attribute__((section(".mysection"))) = 42;
```

If no rule in `SECTIONS` matches that section, it is an **orphan**. The linker still places it, guessing from its flags:

- writable and initialized → RAM, right after `.data`, with its own LMA in FLASH;
- read-only → FLASH.

> **WARNING**
>
> Orphans link silently but escape the script's symbols The link succeeds without warnings, but the orphan lies outside every range defined by the script. Here `tagged` lands exactly at `_edata`, so the copy loop over `_sdata` … `_edata` never initializes it: it reads 0 instead of 42.

2.3 Section Placement of C Declarations

Where a C declaration ends up in memory is decided in two steps:

1. the **compiler** chooses the *input section*, based on the kind of declaration;
2. the **linker script** decides which *memory region* that section goes to.

For declarations at file scope (global):

Global declaration	Section	Region
a function body	`.text`	FLASH
`const int k = 3;`	`.rodata`	FLASH
`"a string literal"`	`.rodata`	FLASH
`int x = 5;`	`.data`	RAM, initial value in FLASH
`int y;` or `int z = 0;`	`.bss`	RAM, zeroed at boot

2.3.1 Special Cases

Case	Where it goes	Why
`int z = 0;`	`.bss`	"starts at zero" is exactly what `.bss` guarantees: no reason to store the zero in flash
`const char *p = "hi";`	`"hi"` → `.rodata` (FLASH), `p` → `.data` (RAM)	two distinct objects: the characters are constant, the pointer is an initialized writable variable
local variable (`int local = 7;` inside a function)	**no section**: the stack	it exists only while the function runs and has no fixed address
`static` local (`static int s = 3;` inside a function)	`.data` (or `.bss`)	it keeps one fixed address for the whole run, like a global; only its name is private

> **WARNING**
>
> `const` pointer vs pointer to `const` In `const char *greeting = "hi";` the `const` applies to the **characters**, not to the pointer. The pointer can still be changed (`greeting = "other";`), so it is an initialized writable variable and goes to `.data` (`D`), while `"hi"` goes to `.rodata` . Only `const char * const greeting = "hi";` makes the pointer itself constant, placing it in `.rodata` (`R`) as well. Reading rule: start from the name and read leftwards: "greeting is a pointer to constant chars".

2.4 Inspecting the Linked Image

Three tools show what the linker actually produced: the map file, `nm` and `objdump` .

2.4.1 Map File

The **map file** (generated with the `-Map=` linker option) lists the *Memory Configuration* (the `MEMORY` regions) and the *Linker script and memory map*: every output section, every input section inside it with its source `.o` , every symbol's address, and the `load address` of `.data` . It also lists *Discarded input sections*.

2.4.2 nm: Listing Symbols

`arm-none-eabi-nm` is the GNU binutils tool that lists the **symbols** of an object file (`.o`) or of the final ELF image (`.axf`). A symbol is a name with an address: functions, global variables, linker-defined symbols such as `_sdata` .

For each symbol it prints:

- the **address**;
- a **letter** giving the kind of section the symbol belongs to;
- the **name**.

```
$ arm-none-eabi-nm -n build/scratch-os.axf
00000044 T limit
20000020 D counter
20000024 D greeting
20000028 B total
```

Letter	Meaning
`T` / `t`	code (`.text`)
`R`	read-only data (`.rodata`)
`D` / `d`	initialized data (`.data`)
`B`	zero-initialized data (`.bss`)
`W`	weak symbol
`A`	absolute symbol, belonging to no section

A lowercase letter means the symbol is local to its file: a `static` local `s` appears as `d s.0` , where the `.N`

`.` suffix distinguishes static locals with the same name in different functions. The `-n` option sorts the symbols by address.

2.4.3 objdump: Section Headers and Raw Bytes

`arm-none-eabi-objdump` shows the content of an ELF file. Two options are used:

Option	Output
`-h`	one line per output section with size, VMA and LMA
`-s -j <section>`	the raw bytes of one section

For an image whose startup file contains only a two-word vector table, `objdump -h` shows `.isr_vector` at `0x0` (8 bytes), `.text` from `0x8`, no `.rodata` line (it is inside `.text`), and `.data` with VMA `0x20000000` and an LMA right after the code in flash, equal to `_sidata`.

Each line of an `-s` dump contains:

- the **address** of its first byte;
- **16 bytes**, in four groups of 4, in memory order;
- the same bytes shown as **text** on the right (`.` for non-printable bytes).

2.5 Worked Example

```
const int limit = 100;
int counter = 5;
int total;
int zero = 0;
const char *greeting = "hi";
char buffer[64];
```

Object	Section	`nm` in `main.o`	`nm` in final image
`limit`	`.rodata`	`R`	`T`
`counter`	`.data`	`D`	`D`
`total`	`.bss`	`B`	`B`
`zero`	`.bss`	`B`	`B`
`greeting`	`.data`	`D`	`D`
`buffer`	`.bss`	`B`	`B`
`"hi"`	`.rodata`	not listed (no name)	not listed

> **INFO**
>
> Why `limit` changes from `R` to `T` `nm` classifies a symbol by the output section it ends up in. The linker script merges every `.rodata` into the `.text` output section, so in the final image `limit` is reported as `T`. The bytes are the same, in the same flash; only the name of the containing section changed.

The bytes of the image confirm the layout:

```
$ arm-none-eabi-objdump -s -j .text build/scratch-os.axf
 0038 00bffde7 4c000000 00000020 64000000  ....L...... d...
 0048 68690000 6d61696e 3a206865 6c6c6f20  hi..main: hello

$ arm-none-eabi-objdump -s -j .data build/scratch-os.axf
 20000020 05000000 48000000                    ....H...
```

Address	Bytes	Meaning
`0x44` (FLASH)	`64 00 00 00`	`limit` = 100 (`0x64`)
`0x48` (FLASH)	`68 69 00`	`"hi"` : `'h'` , `'i'` , terminating `'\0'`
`0x20000020` (RAM)	`05 00 00 00`	`counter` = 5
`0x20000024` (RAM)	`48 00 00 00`	`greeting` = `0x00000048`

> **IMPORTANT**
>
> Little-endian The Cortex-M3 stores the **lowest byte first**. The bytes `05 00 00 00` are therefore the number 5, and `48 00 00 00` the number `0x00000048` .

The last row is the key point: `greeting` holds `0x48` , which is exactly the flash address where `"hi"` starts. **A pointer is simply a variable whose value is an address.**

Variables in `.data` appear in definition order. In this example the first 32 bytes of `.data` (`0x20000000` – `0x2000001F`) hold a string defined earlier in the file, so `counter` and `greeting` follow at `0x20000020` and `0x20000024` .

3 Cortex-M3 Programmer's Model and Exception Model

3.1 Registers and xPSR

The **programmer's model** is the set of **registers visible to code** and to the debugger; all are **32 bits** wide. At this level there are no variables: only registers and memory. A C statement such as `x = x + 1` becomes "load `x` into a register, add 1, store it back".

Register	Name	Role
R0–R12	general purpose	values; a C call passes its first arguments in R0–R3
R13	SP	stack pointer: address of the top of the stack
R14	LR	link register: return address after a function call
R15	PC	program counter: address of the next instruction
—	xPSR	program status: flags, Thumb bit, current exception

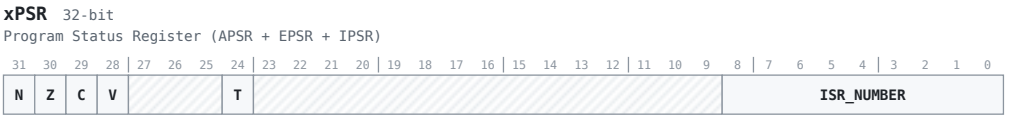
The **stack grows downwards**: a push first does `SP = SP - 4`, then writes at the new SP.

3.1.1 xPSR: the Program Status Register

R0–R15 hold **data or addresses**. xPSR holds neither: it describes the **current state of the processor**. It records the outcome of the last comparison, the instruction set in use, and which exception (if any) is being handled.

The name stands for *Program Status Register*. The `x` indicates that it is the union of three sub-registers, each owning different bits of the same 32-bit word:

Sub-register	Bits	Content	Used for
APSR (Application)	31–28	flags **N Z C V**: negative, zero, carry, overflow	set by `cmp`, read by conditional branches
EPSR (Execution)	24	**T**: Thumb state, always 1 on Cortex-M	instruction set check
IPSR (Interrupt)	8–0	number of the exception being handled (0 = none)	identifying the execution mode



Bits	Field	Description
31	`N`	Negative
30	`Z`	Zero
29	`C`	Carry
28	`V`	Overflow
24	`T`	Thumb state, always 1
8:0	`ISR_NUMBER`	Exception number (0 = Thread mode)

EXAMPLE

Flags in action `cmp r1, r2` computes `r1 - r2`, discards the result and only updates the flags: Z = 1 if the two are equal, C = 1 if `r1 >= r2` (unsigned). The next instruction `bhs done` reads C and jumps if it is 1. This is how every `if` and loop condition works at machine level.

The exception-number field (IPSR) determines the execution **mode**:

Mode	What runs	IPSR (bits 8–0)
Thread mode	normal code: `Reset_Handler`, `main()` and their callees	0
Handler mode	an exception handler	the number of that exception

3.2 Exceptions and Their Priorities

An **exception** is any event that makes the **CPU stop** the current code and run a specific **handler** in Handler mode. An **interrupt** is an exception raised by a hardware device (timer, UART, …).

#	Exception	Raised by
0	(none)	slot holds the initial SP
1	Reset	power-on, reset button
2	NMI	non-maskable interrupt
3	HardFault	any fault with nowhere else to go
4	MemManage	violation of memory-protection rules
5	BusFault	access to an address where nothing answers, or that refuses the access
6	UsageFault	bad instruction use: undefined opcode, Thumb bit 0, divide by 0, …
7–10	reserved	slot must hold 0

#	Exception	Raised by
11	SVCall	the `svc` instruction
12	DebugMonitor	debug events with a debug monitor (unused)
13	reserved	slot must hold 0
14	PendSV	a request raised by software
15	SysTick	the core's built-in timer
16+	IRQs	device interrupts

At reset only **HardFault** is active: MemManage, BusFault and UsageFault are disabled, so every fault **escalates** to HardFault.

When exceptions compete, the more urgent one runs first and can preempt a less urgent handler. On Cortex-M **a lower number means more urgent**. Reset (−3), NMI (−2) and HardFault (−1) have fixed priorities above anything configurable; all other exceptions have a configurable one-byte priority.

> **WARNING**
>
> Lockup If a fault occurs while the CPU is already at HardFault priority (−1), or the fault cannot be reported (e.g. exception entry needs a stack that does not exist), there is nowhere to escalate. The CPU enters **lockup**, reported by QEMU as `Lockup: can't escalate 3 to HardFault (current priority -1)` .

3.3 Reset Sequence and the Vector Table

When the Cortex-M3 leaves reset, **before any software runs**, the hardware performs exactly two reads:

1. the 32-bit word at address `0x0` is loaded into **SP**;
2. the word at address `0x4` is loaded into **PC**, and execution starts there.

There is **no BIOS**, no loader and **no C runtime**. Copying `.data`, zeroing `.bss`, configuring the CPU and calling `main()` are the job of the code at the address stored in the second word.

These two words are the beginning of the **vector table**: a **table of addresses** where slot N holds the address of the handler of exception N. When exception N occurs, the CPU reads slot N and jumps there. Slot 0 is special: it holds the initial SP value, not a code address. The table lives in section `.isr_vector`, which the linker script places first in FLASH, so it lands at address `0x0`, exactly where the CPU reads it at reset.

Cortex-M3 vector table (Scratch OS)

0x0000_003C	[15] **SysTick_Handler** SysTick
0x0000_0038	[14] **PendSV_Handler** PendSV
	⋮ 1 entry (13)
0x0000_0030	[12] **DebugMon_Handler** Debug monitor
0x0000_002C	[11] **SVC_Handler** SVCall
	⋮ 4 entries (7–10)
0x0000_0018	[6] **UsageFault_Handler** UsageFault
0x0000_0014	[5] **BusFault_Handler** BusFault
0x0000_0010	[4] **MemManage_Handler** MemManage
0x0000_000C	[3] **HardFault_Handler** HardFault
0x0000_0008	[2] **NMI_Handler** NMI
0x0000_0004	[1] **Reset_Handler** Reset
0x0000_0000	[0] **_estack** Initial SP (0x20400000)

> **WARNING**
>
> The vector table is in flash, not RAM At reset the table sits at address `0x0`, the start of the **Code/FLASH** region. RAM starts at `0x20000000` and holds only `.data`, `.bss`, heap and stack. The table can be relocated later through VTOR (e.g. into RAM), but this is a deliberate software choice.

The value stored for a handler is its address with **bit 0 set** (e.g. `0x9` for a handler at `0x8`). Instructions are always at even addresses, so bit 0 of a code address carries no information, and ARM reuses it as the **Thumb bit**. The Cortex-M executes only Thumb code and requires this bit to be 1; it checks the bit, copies it into xPSR.T, clears it and jumps to the even address. A table of 16 words (64 bytes) pushes the first code to `0x40`, so word 1 reads `0x41`.

3.4 KEEP and Section Garbage Collection

With the option `-Wl,--gc-sections`, the linker performs **garbage collection of sections**:

1. it starts from the `ENTRY` symbol (`Reset_Handler`);

2. it follows every reference from one section to another (calls, pointers, addresses);
3. it **drops every input section that nothing references**.

This is useful to trim unused code, but it is dangerous for data that **only the hardware reads**. No code references the vector table: the CPU reads it by itself at reset. For the linker it is therefore unused, and it would be discarded.

`KEEP(*(.isr_vector))` marks those input sections as **always kept**, whatever the garbage collector decides.

`--gc-sections`	`KEEP`	Result
off	present or absent	nothing is collected: `KEEP` changes nothing, it only acts as insurance
on	present	`.isr_vector` is kept at `0x0`: the image works
on	absent	`.isr_vector` is discarded: the image does not boot

> **WARNING**
>
> What happens without `KEEP`
>
> 1. `.isr_vector` shrinks to size 0; the map file lists it under *Discarded input sections*.
> 2. `.text` becomes the first section in FLASH, so `Reset_Handler` lands at `0x0`.
> 3. At reset the CPU reads the first two words at `0x0`, which are now **instructions**, and loads them as SP and PC.
> 4. With a meaningless SP and PC, execution faults immediately and the CPU ends in **lockup**.

4 The Startup Code: `boot/startup.s`

The **startup code** (`boot/startup.s`) is the first code the CPU executes after reset, before `main()`. It contains two parts:

- the **vector table**, from which the hardware loads the initial SP and PC;
- `Reset_Handler`, the function that prepares the machine to run C code.

```
// THE COMPLETE STARTUP CODE

/* Assembler settings */
    .syntax unified             /* modern ARM syntax */
    .cpu cortex-m3              /* only M3 instructions */
    .thumb                      /* Thumb code only */

/* 1. Vector table */
    .section .isr_vector,"a",%progbits
    .global isr_vector
    .type isr_vector, %object
isr_vector:
    .word _estack               /* 0: initial SP */
    .word Reset_Handler         /* 1: reset */
    .word NMI_Handler           /* 2: NMI */
    .word HardFault_Handler     /* 3: HardFault */
    .word MemManage_Handler     /* 4: MemManage */
    .word BusFault_Handler      /* 5: BusFault */
    .word UsageFault_Handler    /* 6: UsageFault */
    .word 0                     /* 7: reserved */
    .word 0                     /* 8: reserved */
    .word 0                     /* 9: reserved */
    .word 0                     /* 10: reserved */
    .word SVC_Handler           /* 11: SVCall */
    .word DebugMon_Handler      /* 12: debug monitor */
    .word 0                     /* 13: reserved */
    .word PendSV_Handler        /* 14: PendSV */
    .word SysTick_Handler       /* 15: SysTick */
    .size isr_vector, . - isr_vector

/* Default handlers: weak aliases of Default_Handler */
    .weak      NMI_Handler
    .thumb_set NMI_Handler, Default_Handler
    .weak      HardFault_Handler
    .thumb_set HardFault_Handler, Default_Handler
    .weak      MemManage_Handler
```

```
    .thumb_set MemManage_Handler, Default_Handler
    .weak      BusFault_Handler
    .thumb_set BusFault_Handler, Default_Handler
    .weak      UsageFault_Handler
    .thumb_set UsageFault_Handler, Default_Handler
    .weak      SVC_Handler
    .thumb_set SVC_Handler, Default_Handler
    .weak      DebugMon_Handler
    .thumb_set DebugMon_Handler, Default_Handler
    .weak      PendSV_Handler
    .thumb_set PendSV_Handler, Default_Handler
    .weak      SysTick_Handler
    .thumb_set SysTick_Handler, Default_Handler

/* 2. Reset_Handler */
    .section .text,"ax",%progbits
    .global Reset_Handler
    .type Reset_Handler, %function
    .thumb_func
Reset_Handler:

    /* Step 1: mask interrupts */
    cpsid i                     /* PRIMASK = 1 */

    /* Step 2: set SP */
    ldr r0, =_estack
    mov sp, r0

    /* Step 3: copy .data FLASH -> RAM */
    ldr r0, =_sidata            /* src */
    ldr r1, =_sdata             /* dst */
    ldr r2, =_edata             /* end */
copy_data:
    cmp r1, r2                  /* while (dst < end) */
    bhs copy_data_done
    ldr r3, [r0], #4            /* r3 = *src++ */
    str r3, [r1], #4            /* *dst++ = r3 */
    b copy_data
copy_data_done:

    /* Step 4: zero .bss */
    ldr r1, =_sbss              /* dst */
    ldr r2, =_ebss              /* end */
```

```
    movs r3, #0
zero_bss:
    cmp r1, r2                  /* while (dst < end) */
    bhs zero_bss_done
    str r3, [r1], #4            /* *dst++ = 0 */
    b zero_bss
zero_bss_done:

    /* Step 5: VTOR = &isr_vector */
    ldr r0, =0xE000ED08
    ldr r1, =isr_vector
    str r1, [r0]

    /* Step 6: SHCSR |= bits 16-18 */
    ldr r0, =0xE000ED24
    ldr r1, [r0]
    orr r1, r1, #0x00070000
    str r1, [r0]

    /* Step 7: CCR |= DIV_0_TRP */
    ldr r0, =0xE000ED14
    ldr r1, [r0]
    orr r1, r1, #(1 << 4)
    str r1, [r0]

    /* Step 8: priorities (SHPR1-3) */
    ldr r0, =0xE000ED18
    movs r1, #0x00
    strb r1, [r0, #0]           /* MemManage */
    strb r1, [r0, #1]           /* BusFault */
    strb r1, [r0, #2]           /* UsageFault */
    movs r1, #0x80
    strb r1, [r0, #7]           /* SVCall */
    movs r1, #0xE0
    strb r1, [r0, #10]          /* PendSV */
    strb r1, [r0, #11]          /* SysTick */

    /* Step 9: unmask interrupts */
    cpsie i                     /* PRIMASK = 0 */

    /* Step 10: call main, then hang */
    bl main
hang:
```

```
    b hang

    .ltorg                      /* literal pool */
    .size Reset_Handler, . - Reset_Handler

/* 3. Default_Handler */
    .global Default_Handler
    .type Default_Handler, %function
    .thumb_func
Default_Handler:
    b Default_Handler           /* for (;;) {} */
    .size Default_Handler, . - Default_Handler
```

The order of the steps is dictated by dependencies: nothing may interrupt a half-built system, C needs a stack, globals must hold their values before any C code reads them, and the configuration must be complete before interrupts are accepted.

Step	Action	Reason
1	mask interrupts (`cpsid i`)	nothing may interrupt a half-built system
2	set SP	C functions need a stack
3	copy `.data` FLASH → RAM	initialized globals get their values
4	zero `.bss`	C guarantees uninitialized globals start at 0
5–8	configure the CPU	vector table address, fault handlers, divide-by-zero trap, priorities
9	unmask interrupts (`cpsie i`)	setup is complete
10	`bl main` , then loop forever	hand over to C; on bare metal there is nothing to return to

It works in sequence:

1. **Hardware**: at reset the CPU loads SP from word 0 and PC from word 1 of the vector table, so execution starts at `Reset_Handler` .
2. **Stack**: `Reset_Handler` sets SP to `_estack` , the top of RAM.
3. **`.data`** : it copies the initial values of the initialized variables from FLASH (`_sidata`) to RAM (`_sdata` … `_edata`).
4. **`.bss`** : it fills `_sbss` … `_ebss` with zeros.
5. **CPU configuration**: it sets up the vector table address, fault handlers and priorities.
6. **`main()`** : it calls `main` and, if `main` ever returns, loops forever.

All the addresses used in steps 2–4 are symbols defined by the linker script (chapter 2). The two work at different times: the linker script is used only at **build time** to place sections and define those symbols, and it is never executed; the startup code runs on the CPU at **run time** and uses them as loop bounds. The full code and each step are covered in chapter 5.

> **INFO**
>
> Why assembly When the startup code begins, the stack has not been set up yet and `.data` and `.bss` are not initialized. C code takes all of these for granted, so it cannot be trusted until the startup code has finished. On a desktop this job is done by **crt0**, the C library's startup code; on bare metal it is excluded (`-nostartfiles`) and written by hand.

Writing the startup code needs only a small subset of Thumb instructions, described below.

4.1 Load/Store Architecture

ARM arithmetic **operates only on registers**; memory is accessed only by **load** (`ldr` , memory to register) and **store** (`str` , register to memory) instructions.

Assembly	C equivalent
`ldr r3, [r0]`	`r3 = *r0;`
`str r3, [r1]`	`*r1 = r3;`
`ldr r3, [r0], #4`	`r3 = *r0; r0 += 4;` (i.e. `r3 = *src++;`)
`str r3, [r1], #4`	`*r1 = r3; r1 += 4;` (i.e. `*dst++ = r3;`)
`strb r1, [r0, #7]`	`((uint8_t *)r0)[7] = r1;` (one byte at offset 7)

Square brackets mean "the memory at this address". `[r0], #4` is **post-increment**: the address in `r0` is used first, then 4 is added, so a loop walks through memory one word at a time.

4.2 Constants, Labels, and Branches

Assembly	C equivalent
`ldr r0, =_estack`	`r0 = (uint32_t)&_estack;` (any 32-bit value)
`mov sp, r0`	`sp = r0;` (register to register)
`movs r3, #0`	`r3 = 0;` (small constants only)
`copy_data:`	a label: a name for this address
`b copy_data`	`goto copy_data;`
`bl main`	`main();` (branch with link: return address saved in LR)
`cmp r1, r2`	compute `r1 - r2` , keep only the N Z C V flags
`bhs done`	`if (r1 >= r2) goto done;` (higher or same, unsigned)
`orr r1, r1, #0x10`	`r1 |= 0x10;`
`cpsid i` / `cpsie i`	`__disable_irq()` / `__enable_irq()`

An instruction is only 16 or 32 bits long and cannot embed an arbitrary 32-bit constant. For `ldr rX, =value` the

assembler stores the constant in a nearby table, the **literal pool**, and emits a PC-relative load, which the debugger shows as `ldr r0, [pc, #92]`. The `.ltorg` directive places the pool after the last instruction, where the CPU never executes it.

A C `while` loop becomes "test at the top, jump back at the bottom":

```
loop:   cmp r1, r2          @ while (r1 < r2) {
        bhs done            @
        ...                 @     body
        b   loop            @ }
done:
```

4.3 Assembler Directives

Lines starting with a dot produce no instructions; they steer the assembler.

```
    .section .isr_vector,"a",%progbits   @ following content goes into .isr_vector
    .global isr_vector                   @ name visible to the linker
isr_vector:
    .word _estack                        @ one 32-bit value
    .word Reset_Handler

    .section .text,"ax",%progbits
    .global Reset_Handler
    .thumb_func                          @ next label is Thumb code: bit 0 = 1
Reset_Handler:
```

The equivalent vector table in C is:

```
__attribute__((section(".isr_vector")))
const uint32_t isr_vector[] = { (uint32_t)&_estack, (uint32_t)Reset_Handler };
```

Directive	Effect
`.section name,"flags"`	selects the section; `a` = occupies space in the image, `x` = executable
`.global name`	exports the name to other files and the linker (otherwise it behaves like a `static` C name)
`.thumb_func`	marks the next label as Thumb code, so its address is stored with bit 0 set
`.word X`	stores the 32-bit value X
`.type`, `.size`	tell `nm` and gdb whether a name is code or data, and its size
`.weak name`	declares a weak symbol, overridable by a normal definition
`.thumb_set a, b`	makes `a` an alias of `b`, marked as Thumb code

Directive	Effect
`.ltorg`	emits the literal pool here

5 Reset Handler: From Reset to main()

`Reset_Handler` is the function the CPU executes **first** after every reset. Its address is **stored in slot 1 of the vector table**: at reset the hardware loads that word into PC, so execution starts here, before any C code.

On bare metal there is no operating system and no C library to prepare the machine. `Reset_Handler` takes their place: it brings the CPU and the memory into the state that C code assumes, configures how the CPU reacts to errors, and only then calls `main()`. Its work is organized in two jobs, carried out in order:

Job	Steps	Without it
Make C code safe to run	set the stack, copy `.data`, zero `.bss`	globals hold wrong values, function calls have no valid stack
Configure the CPU	vector table address, separate fault handlers, divide-by-zero trap, priorities	errors pass silently or all end up in the same HardFault

EXAMPLE

A startup that only calls `main` Consider a minimal `Reset_Handler` that executes only `bl main`, and a global `char data_message[] = "hello";`:

- the linker places the bytes `"hello"` in **FLASH** (LMA) and reserves space for `data_message` in **RAM** (VMA);
- nobody copies the bytes from FLASH to RAM;
- `main` reads `data_message` from **RAM**, where the copy was never written.

The result depends on what RAM contains at power-on. QEMU fills RAM with zeros, so the first byte is `'\0'` and the string reads as **empty**. Real hardware leaves random content, so the string reads as **garbage**. The program compiles and runs, but the data is wrong: this is the kind of silent error the copy loop prevents.

5.1 Weak Default Handlers

The vector table has 16 slots, and the CPU jumps to whatever address slot N contains when exception N occurs. Every slot must therefore point to valid code, even for exceptions the program does not handle yet; reserved slots simply hold `0`. Instead of writing nine real handlers, the startup code uses two pieces:

- **`Default_Handler`**: a single catch-all function, an infinite loop (`for (;;) {}`);
- **weak names**: every handler name in the table is declared `.weak` and, by default, is an alias of `Default_Handler` (same address).

A **weak** symbol is a default that gives way: if another file defines a normal (strong) function with the same name, the linker uses that one and drops the weak alias. A handler is installed simply by writing a C function with the right name, without touching the assembly. The C equivalent of each weak name is:

```
void HardFault_Handler(void) __attribute__((weak, alias("Default_Handler")));
```

With only `UsageFault_Handler` defined in `main.c`:

Slot	Name	Defined in C?	CPU jumps to	`nm`
3	`HardFault_Handler`	no	`Default_Handler`	`W`, same address as `Default_Handler`
6	`UsageFault_Handler`	yes	the C function	`T`, its own address
15	`SysTick_Handler`	no	`Default_Handler`	`W`, same address as `Default_Handler`

> **INFO**
>
> Why an infinite loop An unexpected exception is a bug. Freezing at a known place lets a debugger show immediately that PC is in `Default_Handler`, instead of letting the CPU run off into random memory.

5.2 Interrupt Masking and Stack Setup

`cpsid i` sets **PRIMASK**, a 1-bit core register (not memory-mapped): while it is 1, every exception with configurable priority is ignored; only NMI and HardFault get through. Interrupts arriving while masked are not lost: they remain **pending** and run as soon as `cpsie i` clears PRIMASK. PRIMASK is 0 at reset; the boot code sets it first and clears it only after the vector table, handlers and priorities are in place.

Instruction	PRIMASK	C (vendor headers)
`cpsid i`	1: masked	`__disable_irq()`
`cpsie i`	0: accepted	`__enable_irq()`

SP is set explicitly even though the hardware already loaded it from vector word 0, because `Reset_Handler` is not always reached through a real reset: a bootloader or debugger may jump to it with SP holding any value. SP is a special register, so the address is first loaded into `r0` with `ldr` and then moved with `mov sp, r0`.

> **WARNING**
>
> Failure modes of a wrong SP With SP = `0x30000000` (outside RAM), the first push in `main` writes to a non-existent address and faults; reporting the fault requires stacking registers, which faults again, and the CPU enters lockup. With SP = `0x20000001`, the CPU silently clears the two low bits (SP is always word-aligned), so the stack starts at the bottom of RAM and grows *below* it, into an unmapped range where QEMU ignores writes: every call fails silently and nothing is printed.

5.3 Initializing .data and .bss

The `.data` copy translates `while (dst < &_edata) *dst++ = *src++;` with `r0` = `src` (`_sidata`), `r1` = `dst` (`_sdata`) and `r2` = end (`_edata`). It copies whole words, which is safe because the linker script aligns the start and end of `.data` to 4 bytes. Stepping through the loop in a debugger shows `r1` climbing `0x20000000`, `0x20000004`, … until it equals `&_edata`; only then does the string appear in RAM.

The `.bss` loop has the same shape without a source pointer: `while (dst < &_ebss) *dst++ = 0;`. When `.bss` is empty, `_sbss` equals `_ebss` and the loop exits immediately. Relying on the emulator zeroing RAM is incorrect: real SRAM is random at power-on. After these two loops, `Reset_Handler` provides exactly what a C runtime startup (crt0) provides.

5.4 System Control Block Registers

Most core settings are 32-bit registers mapped at fixed addresses in the **System Control Space** (`0xE000E000` – `0xE000EFFF`). The group controlling exceptions is the **System Control Block (SCB)**. They are accessed with ordinary loads and stores:

```
#define SCB_SHCSR (*(volatile uint32_t *)0xE000ED24)
SCB_SHCSR |= (1u << 16);
```

```
    ldr r0, =0xE000ED24   @ address
    ldr r1, [r0]          @ read
    orr r1, r1, #0x10000  @ modify
    str r1, [r0]          @ write
```

This is the **read-modify-write** pattern: read the current value, set only the required bits, write back, so all other bits keep their value. `volatile` forces the compiler to perform every access, in order: it may neither skip a read ("the value cannot have changed") nor remove a write ("nobody reads it"), assumptions that are false for hardware registers. As an example, the CPUID register at `0xE000ED00` reads `0x410fc231` : implementer ARM (`0x41`), part Cortex-M3 (`0xc23`).

Register	Address	Role	Value after boot
VTOR	`0xE000ED08`	address of the vector table	`0x00000000`
CCR	`0xE000ED14`	what counts as a fault	`0x00000210`
SHPR1–3	`0xE000ED18` – `0xE000ED20`	system handler priorities	`0x00000000` `0x80000000` `0xe0e00000`
SHCSR	`0xE000ED24`	enable and status of system handlers	`0x00070000`

5.4.1 VTOR: Vector Table Location

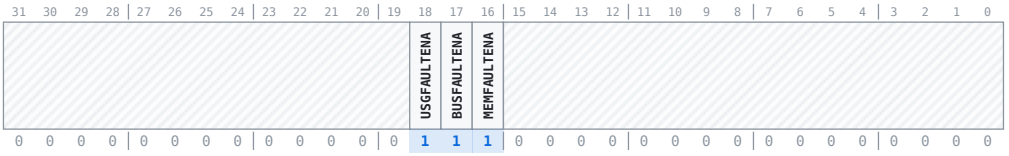
On an exception the CPU reads the handler from **VTOR + 4 × N**, not from `4 × N` . VTOR resets to `0x00000000` , which is why the table must be at address 0 at reset. Bits 31–7 hold the table address, which must be aligned to at least 128 bytes. The whole register is written, so no read-modify-write is needed. Writing it is redundant when the table is already at 0, but it makes the boot code correct wherever the image is placed: a **bootloader** at address 0 may start an application stored higher in flash, which then points VTOR at its own table; or a program may copy its table into RAM to swap handlers at run time.

5.4.2 SHCSR: Separate Fault Handlers

Bit	Name	When set
16	MEMFAULTENA	MemManage faults run `MemManage_Handler`
17	BUSFAULTENA	bus faults run `BusFault_Handler`

Bit	Name	When set
18	USGFAULTENA	usage faults run `UsageFault_Handler`

SHCSR 32-bit = 0x00070000
System Handler Control and State Register (0xE000ED24), value after boot



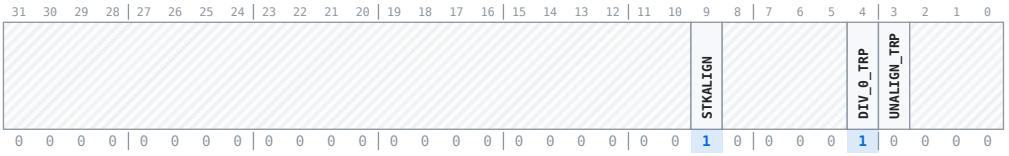
Bits	Field	Value	Description
18	`USGFAULTENA`	1	UsageFault handler enabled
17	`BUSFAULTENA`	1	BusFault handler enabled
16	`MEMFAULTENA`	1	MemManage handler enabled

At reset these bits are 0, so every fault escalates to HardFault and a single handler cannot tell what went wrong. Setting bits 16–18 (`|= 0x00070000`) routes each fault to its own handler. The other SHCSR bits report status (active or pending handlers), so the register must be updated with read-modify-write, never overwritten.

5.4.3 CCR: Divide-by-Zero and Alignment Traps

Bit	Name	Meaning	Setting
3	UNALIGN_TRP	trap unaligned word/halfword accesses	left off
4	DIV_0_TRP	trap integer divide by 0 as a UsageFault	set
9	STKALIGN	8-byte-align the stack on exception entry	already 1 at reset

CCR 32-bit = 0x00000210
Configuration and Control Register (0xE000ED14), value after boot



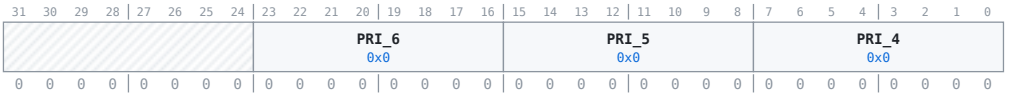
Bits	Field	Value	Description
9	`STKALIGN`	1	8-byte stack alignment on exception entry
4	`DIV_0_TRP`	1	Trap integer divide by zero
3	`UNALIGN_TRP`	0	Trap unaligned accesses

Without DIV_0_TRP, an integer division by zero on Cortex-M3 does not fault: `sdiv` / `udiv` silently return **0**, and the program continues with a wrong value. Setting bit 4 turns this silent bug into a UsageFault. Unaligned accesses are legal (only slower) on Cortex-M3; trapping them would expose sloppy pointer casts but can break correct compiler- or library-generated code, so the trap stays off. Read-modify-write is essential here, since STKALIGN is already 1 and must be preserved: the resulting value `0x210` is bit 4 plus bit 9.

5.4.4 SHPR: System Handler Priorities

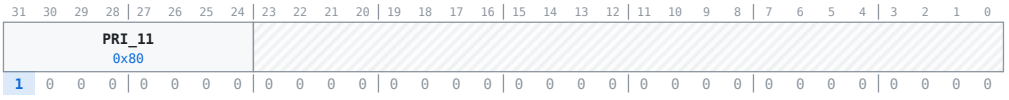
Register	Byte 0	Byte 1	Byte 2	Byte 3
SHPR1 `0xE000ED18`	MemManage	BusFault	UsageFault	—
SHPR2 `0xE000ED1C`	—	—	—	SVCall
SHPR3 `0xE000ED20`	—	—	PendSV	SysTick

SHPR1 32-bit = **0x00000000**
System Handler Priority Register 1 (0xE000ED18)



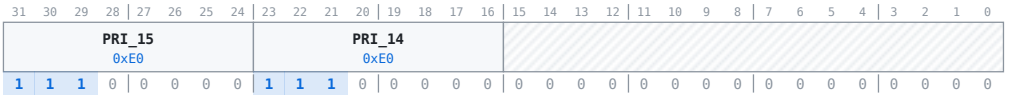
Bits	Field	Value	Description
23:16	`PRI_6`	0x0 (0)	UsageFault priority
15:8	`PRI_5`	0x0 (0)	BusFault priority
7:0	`PRI_4`	0x0 (0)	MemManage priority

SHPR2 32-bit = **0x80000000**
System Handler Priority Register 2 (0xE000ED1C)



Bits	Field	Value	Description
31:24	`PRI_11`	0x80 (128)	SVCall priority

SHPR3 32-bit = **0xE0E00000**
System Handler Priority Register 3 (0xE000ED20)



Bits	Field	Value	Description
31:24	`PRI_15`	0xE0 (224)	SysTick priority
23:16	`PRI_14`	0xE0 (224)	PendSV priority

Each priority is a separate byte, so it is written with a byte store (`strb`) and needs no read-modify-write. Treating SHPR1–3 as a 12-byte array at `0xE000ED18` , the indices are 0–2 (faults), 7 (SVCall) and 10–11 (PendSV, SysTick). The chosen policy is faults most urgent (`0x00`), SVCall intermediate (`0x80`), PendSV and SysTick least urgent (`0xE0`). Real chips implement only the **upper 3 or 4 bits** of each priority byte (lower bits read as 0), so values must differ in their top bits. Reading the words as little-endian, `0x80000000` has `0x80` in byte 3 and `0xe0e00000` has `0xe0` in bytes 2 and 3.

5.5 Fault Handlers in C and the Divide-by-Zero Trap

```c
void UsageFault_Handler( void )
{
    semihost_write0( "UsageFault_Handler: a usage fault happened\n" );
    for ( ;; ) { }
}

int main( void )
{
    volatile uint32_t numerator = 42;
    volatile uint32_t zero = 0;
    volatile uint32_t result = numerator / zero;
    (void)result;
    /* ... */
}
```

`UsageFault_Handler` is a strong definition, so it replaces the weak alias; `HardFault_Handler`, `MemManage_Handler` and `BusFault_Handler` follow the same pattern. Each handler prints its name and loops forever, because a fault is a bug with no sensible way to continue. The divisor is `volatile`: otherwise the compiler would see the constant 0 and compute or remove the division at build time, and no divide instruction would execute. `(void)result;` only suppresses the unused-variable warning.

The outcome isolates the role of each configuration step:

Configuration	Output
SHCSR bits and DIV_0_TRP set	`UsageFault_Handler: a usage fault happened`
SHCSR bits not set	`HardFault_Handler: …` (the fault escalates, its kind is lost)
DIV_0_TRP not set	no fault: the division returns 0 and `main` continues

5.6 Additional Startup Work on Real Chips

QEMU's CPU runs at full speed out of reset. On a real microcontroller the startup code usually does more, often in a vendor function `SystemInit()` called from `Reset_Handler`:

- **Clock and PLL**: the chip starts on a slow internal oscillator, and a PLL multiplies the frequency up to full speed.
- **Flash wait states**: flash is slower than a fast CPU, so the flash controller must be told to insert wait cycles *before* the clock is raised, or the CPU reads garbage.
- **Watchdog**: a timer that resets the chip unless software periodically feeds it; on some chips it runs from reset, so boot code must feed or disable it.
- **FPU**: on a Cortex-M4F the floating-point unit is off at reset and must be enabled in **CPACR** (`0xE000ED88`), otherwise the first float instruction raises a UsageFault.
- **C++**: static constructors listed in `.init_array` must be called before `main`.

5.7 Verifying Boot State with gdb

A **debugger** controls a running program: it stops it at a breakpoint or before the first instruction, steps it one machine instruction or one C line at a time, inspects and modifies registers and memory, and resumes it. On a microcontroller, where a crash often produces no message at all, it is frequently the only way to see what the CPU does. The ARM build of gdb is `arm-none-eabi-gdb` (`gdb-multiarch` on Linux).

Debugging is **remote**: the program runs inside QEMU, and gdb connects over TCP. QEMU is started with the normal run command plus `-S` (freeze the CPU before its first instruction) and `-gdb tcp::1234` (start a gdb server on port 1234). gdb reads the ELF only for symbols and debug information; QEMU already holds the code.

```
qemu-system-arm -machine mps2-an385 -monitor null -semihosting \
    --semihosting-config enable=on,target=native \
    -kernel build/scratch-os.axf -serial stdio -nographic \
    -S -gdb tcp::1234
```

Command	Effect
`target remote :1234`	connect to the gdb server
`info registers sp pc`	show registers (`info registers` shows all)
`x/2wx 0`	examine 2 words in hex at address 0
`x/s &data_message`	examine memory as a C string
`p/x $r1`	print a register or C expression in hex
`display/i $pc`	show the next instruction after every step
`stepi` (`si`)	execute one machine instruction
`break main` (`b`)	stop when execution reaches a symbol or label
`continue` (`c`)	run to the next breakpoint

Attached before the first instruction, gdb already shows SP = `0x20400000` and PC = `Reset_Handler`, while `x/2wx 0` shows `0x20400000 0x00000041` (or `0x9` with a two-word table): no instruction has executed, so the hardware itself loaded both registers from the vector table, clearing the Thumb bit for PC. At `break main`, the SCB registers show the values in the table above and `info registers primask` reads 0 (it reads 0 at reset and 1 after `cpsid i`).

6 Freestanding C and Semihosting

6.1 Hosted and Freestanding Environments

The C standard defines two environments:

Aspect	Hosted	Freestanding
Example	desktop program	bare-metal Cortex-M3 image
Below the program	a C library (libc) and an OS	nothing
`printf`, `malloc`, `fopen`, `exit`	available	not available
Who starts `main()`	libc startup code, invoked by the OS	the application's own startup code

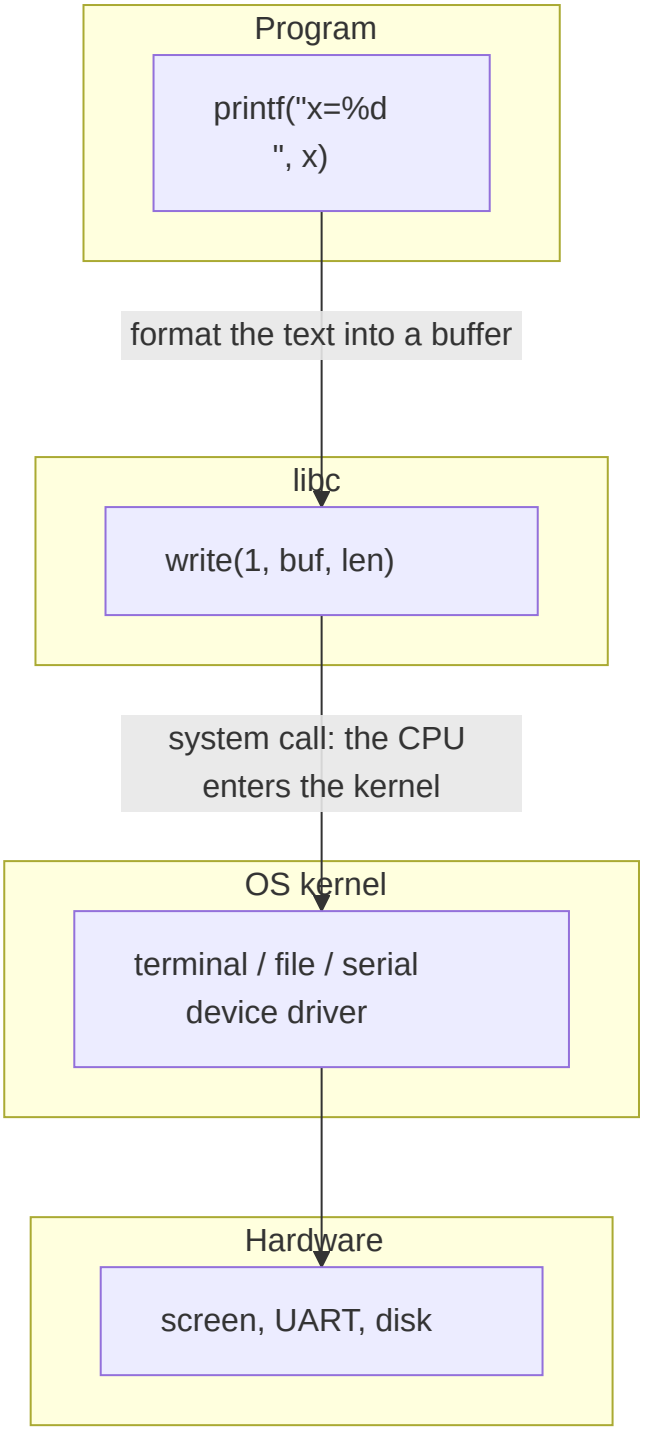
A freestanding environment still provides the headers that need **no run-time code**, only types, constants and macros supplied by the compiler:

Header	Provides
`stdint.h`	`uint32_t`, `int8_t`, …
`stddef.h`	`size_t`, `NULL`, `offsetof`
`stdbool.h`	`bool`, `true`, `false`
`stdarg.h`	`va_list` for variadic functions
`limits.h`, `float.h`	`INT_MAX`, `FLT_EPSILON`, …

There is no `stdio.h`, `stdlib.h` or `string.h`: no printing, no `malloc`, no `exit`, not even `memcpy`. This is why the boot code copies `.data` with its own loop.

6.2 The Layers Under printf

On a desktop, `printf` looks like a **single function** that writes to the screen. In reality it is the top of a stack of layers, and **most of the work happens in layers that a bare-metal system does not have**:



`printf` itself only **formats**: it turns `"x=%d"` and `42` into the characters `x=42` in a buffer. **Getting those characters out of the machine** is the **OS's** job, reached through the `write()` system call. Many other libc functions have the same structure: a portable part on top, and a dependency on something underneath.

libc feature	Requires underneath
`printf`, `puts`	`write()`: a way to send bytes to a device
`malloc`, `free`	a heap region, grown with `sbrk()`
`exit`	something to return to

libc feature	Requires underneath
`main()` being called	crt0: sets up stack, `.data`, `.bss`, then calls `main`

6.2.1 libc on Bare Metal

Embedded C libraries such as **newlib** keep the portable part (formatting, string handling, data structures) and leave the bottom functions (`write`, `sbrk`, `exit`, …) as **hooks**: empty stubs that must be implemented for each board.

This course links **no libc at all**, for three reasons:

Reason	Explanation
Nothing to put under the hooks	there is no kernel, no `write()`, no heap manager yet
crt0 is not needed	`Reset_Handler` already does its work
No hidden code	the image contains only the startup file, the application and a few compiler helpers

> **INFO**
>
> What replaces `printf` Without libc, output is produced by the program itself: in this chapter through **semihosting** (6.4), and from week 3 through a UART driver and the course's own `kprintf` / `printf`.

6.3 Build Flags for a Freestanding Image

```
CFLAGS := -mthumb -mcpu=cortex-m3 -nostartfiles -ffreestanding -std=gnu17
LDFLAGS += -nostdlib -lgcc
```

Flag	Meaning
`-ffreestanding`	GCC must not assume a libc or OS exists (e.g. that a function named `printf` behaves like the standard one)
`-nostartfiles`	do not link the toolchain's startup code (crt0)
`-nostdlib`	do not link any standard library or startup file (no `-lc`)
`-lgcc`	add back libgcc, the compiler's own helper routines

`libgcc` is kept because the compiler sometimes turns a plain C operator into a **call** to a helper when the CPU has no single instruction for it. These helpers need neither OS nor libc, and dropping them would make harmless code fail to link.

C code	CPU	GCC emits
`a / b` (32-bit)	Cortex-M0 (no divide instruction)	`bl __aeabi_idiv`
`a / b` (32-bit)	Cortex-M3	`sdiv` (hardware)

C code	CPU	GCC emits
`x / y` (64-bit)	Cortex-M3	`bl __aeabi_uldivmod`

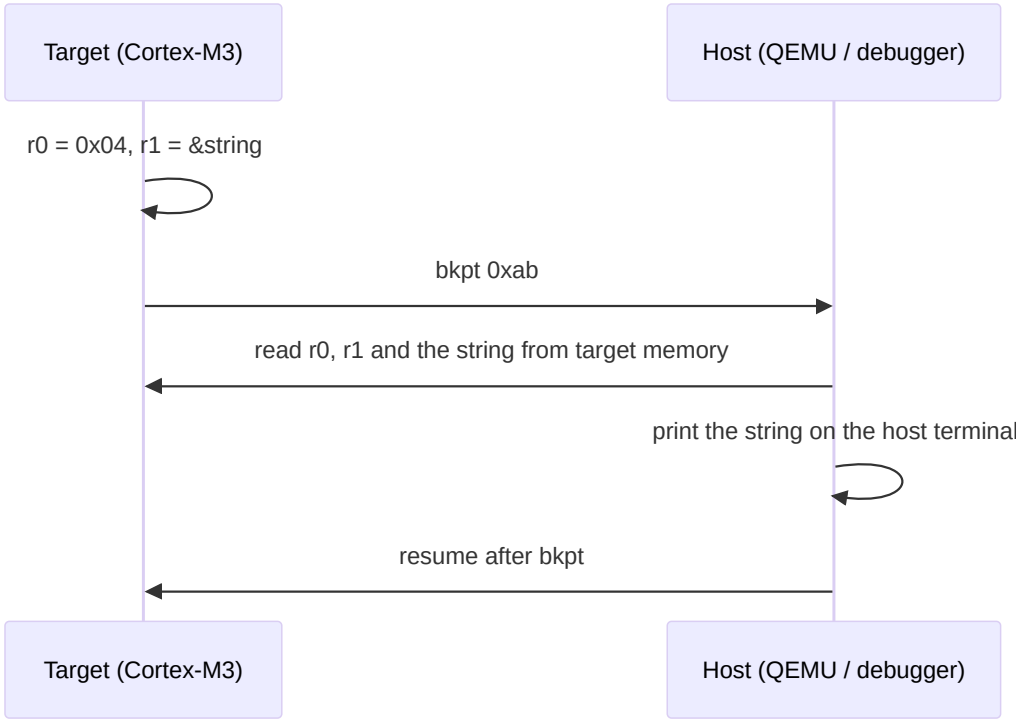
Calling `printf` in such a build always fails: with a toolchain that ships no C library, compilation stops with `stdio.h: No such file or directory`; with a toolchain that ships newlib, the header is found but the link fails with `undefined reference to 'printf'` because of `-nostdlib`.

6.4 Semihosting Output via bkpt 0xab

Without libc and without a UART driver, the program has no way of its own to print anything. **Semihosting** solves this by asking the **host** (the PC running QEMU or the debugger) to perform the I/O on the target's behalf. It is the mechanism that produces every message printed so far.

The request follows a fixed convention:

Register / instruction	Content
`r0`	operation code: `0x04` = `SYS_WRITE0`, print a NUL-terminated string
`r1`	argument: the address of the string
`bkpt 0xab`	a breakpoint with the special immediate `0xab`: "semihosting request"



In C, the request is wrapped in a small function:

```
static void semihost_write0( const char *message )
{
    register uint32_t    r0 __asm__( "r0" ) = 0x04;    /* SYS_WRITE0 */
    register const char *r1 __asm__( "r1" ) = message; /* string address */
```

```
    __asm__ volatile ( "bkpt 0xab" : : "r"( r0 ), "r"( r1 ) : "memory" );
}
```

Element	Meaning
`register … __asm__( "r0" )`	GCC extension that forces a variable into a specific register; standard C gives no control over registers, but the convention requires `r0` and `r1`
`__asm__ volatile ( "bkpt 0xab" … )`	inserts an instruction that no C statement can express; `volatile` prevents the compiler from removing or moving it
`: "r"( r0 ), "r"( r1 )`	the instruction uses these two registers as inputs
`"memory"` clobber	the instruction may read memory (the string), so every pending store must complete before it

WARNING

Only with a host listening Semihosting works only when an emulator or debugger intercepts the `bkpt` (`-semihosting` in QEMU). On a standalone board with no debugger attached, `bkpt` simply halts the CPU.

WARNING

Multiple instructions in one `__asm__` C concatenates adjacent string literals, so `"ldr r2, [r0]" "ldr r3, [r1]"` reaches the assembler as the single line `ldr r2, [r0]ldr r3, [r1]` and fails with `garbage following instruction` . Each instruction must end with `\n` (commonly `\n\t`) inside the string.

INFO

Semihosting is a trap The program stops its own work, control passes to a more powerful entity outside the CPU, which performs the job and then returns control. This is exactly the shape of a **system call** (chapter 7), with the host playing the role of the kernel.

7 Privileged Execution and the SVC System Call

7.1 Privileged and Unprivileged Thread Mode

The Cortex-M3 implements in hardware the kernel-mode/user-mode separation of operating systems, combined with the two execution modes:

CPU mode	Privilege	Typical code
Thread	privileged	`Reset_Handler`, `main` after reset (the kernel)
Thread	unprivileged	application code after `CONTROL.nPRIV = 1`
Handler	privileged	every exception handler

> **IMPORTANT**
>
> Only three combinations nPRIV affects **Thread mode only**: Handler mode is always privileged. An exception therefore always runs privileged, and returns to Thread mode with the privilege it had before.

After reset the CPU is in privileged Thread mode, which is why `Reset_Handler` can write VTOR, SHCSR and CCR. Unprivileged Thread mode is the equivalent of user mode: the mode for code a kernel would not trust.

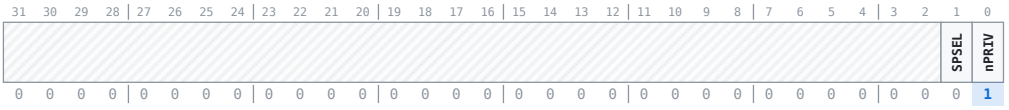
7.2 The CONTROL Register

Thread-mode privilege is selected by **CONTROL**, a special core register (not memory-mapped, like PRIMASK):

Bit	Name	Meaning
0	nPRIV	0 = privileged, 1 = unprivileged (Thread mode only)
1	SPSEL	selects which stack pointer Thread mode uses

CONTROL 32-bit = 0x00000001

CONTROL register after the privilege drop



Bits	Field	Value	Description
1	`SPSEL`	0 → MSP	Thread-mode stack pointer 0 = MSP, 1 = PSP
0	`nPRIV`	1 → unprivileged	Thread-mode privilege 0 = privileged, 1 = unprivileged

Special registers are not reachable with `ldr` / `str`; they are read with `mrs r0, control` (`r0 = CONTROL;`) and written with `msr control, r0` (`CONTROL = r0;`).

7.3 Dropping Privilege

```
    mrs r0, control     @ r0 = CONTROL
    orr r0, r0, #1      @ set nPRIV
    msr control, r0     @ now unprivileged
    isb                 @ discard instructions fetched under the old rules
```

This is a read-modify-write on a special register (`CONTROL = CONTROL | 1;`). The `isb` (instruction synchronization barrier) is required because the CPU prefetches instructions: some may already have been fetched under the privileged rules, and `isb` flushes and refetches them under the new ones.

> **IMPORTANT**
>
> A one-way street Writing CONTROL is itself privileged: an unprivileged `msr control, r0` that clears nPRIV is silently **ignored**. The only ways back to privileged execution are a **reset** or an **exception**, whose handler runs in Handler mode, always privileged.

7.4 Restrictions on Unprivileged Code

Attempt	Result
read/write the System Control Space (`0xE000E000` …): SysTick, NVIC, SCB	BusFault
`msr control, r0` to regain privilege	ignored
`cpsid i` / `cpsie i`	ignored
semihosting `bkpt 0xab`	refused by QEMU by default, so it faults

(The NVIC is the core unit that manages device interrupts, located in the same System Control Space as the SCB.) Ordinary RAM, flash and the program's own code remain accessible: restricted code cannot **reconfigure** the machine or silence it by masking interrupts. A semihosting call after the privilege drop therefore never prints: the refused `bkpt` becomes a fault with no dedicated handler and escalates to HardFault, whose handler *can* print because handlers run privileged.

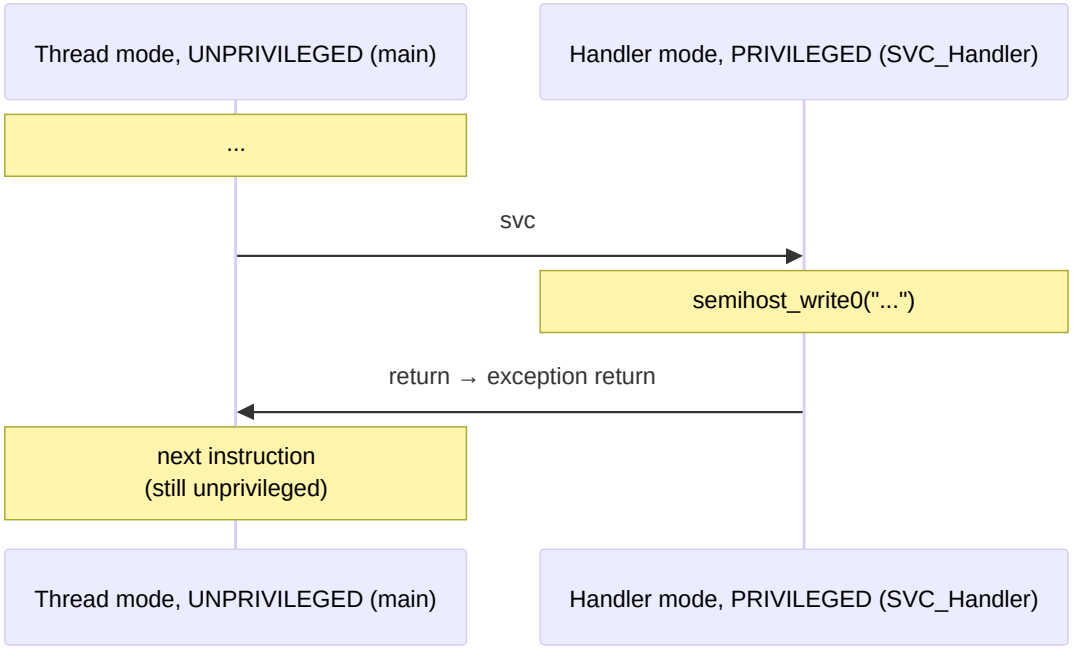
7.5 SVC as a Controlled Entry into Privileged Code

```
void SVC_Handler( void )
{
    semihost_write0( "SVC_Handler: privileged code printing on behalf of unprivileged code\n"
);
}

/* in main(), after the privilege drop */
__asm__ volatile ( "svc #0" : : : "memory" );
```

```
volatile uint32_t *systick_ctrl = ( volatile uint32_t * ) 0xE000E010;
*systick_ctrl = 0;   /* unprivileged store to the System Control Space */
```

`svc #n` (supervisor call) is permitted in unprivileged code. It raises **SVCall** (exception 11), whose handler is in slot 11 of the vector table. Like any exception it runs in Handler mode, hence **privileged**. When the handler returns, execution resumes at the instruction after `svc`, in Thread mode and **still unprivileged**:



Privilege is **borrowed** only for the duration of the handler, and only code placed in the handler runs with it: the caller can request a service but cannot choose what executes. The immediate `#n` is encoded in the instruction; real kernels use such a request number, plus arguments in registers, to select the service.

`SVC_Handler` returns normally instead of looping, because `svc` is a deliberate request, not an error. A plain C function can serve as a handler because on exception entry the Cortex-M automatically saves the registers a C function may clobber (R0–R3, R12, LR, PC, xPSR) and restores them on return. If `SVC_Handler` is not defined, `svc` lands in the weak `Default_Handler` and the program hangs silently.

The final store to SysTick's control register (`0xE000E010`) is refused by the System Control Space and raises a **BusFault**, which reaches `BusFault_Handler` because SHCSR enabled bus faults. Inside `SVC_Handler`, CONTROL still reads `0x1`: the handler is privileged because it runs in Handler mode, not because nPRIV changed.

7.6 Comparison with an OS System Call

Aspect	Linux on a desktop	Bare-metal Cortex-M3
Restricted code	user-mode process	`main` after nPRIV = 1
Trap instruction	`syscall` (x86-64), `svc` (ARM)	`svc #0`
Privileged side	the kernel	`SVC_Handler`

Aspect	Linux on a desktop	Bare-metal Cortex-M3
Service	`write()` puts bytes on the terminal	print via semihosting

The mechanism is identical at every scale: the restricted side cannot do the job itself, so it traps into code that can, and that code decides what to do. This is the core of every operating system's protection model: restricted code cannot touch the machine directly and must go through a single, controlled entry point.